

Intro to Configuring Python Applications

 [_ \(https://github.com/learn-co-curriculum/python-p3-intro-to-configuring-applications\)](https://github.com/learn-co-curriculum/python-p3-intro-to-configuring-applications)  [_ \(https://github.com/learn-co-curriculum/python-p3-intro-to-configuring-applications/issues/new\)](https://github.com/learn-co-curriculum/python-p3-intro-to-configuring-applications/issues/new)

Learning Goals

- Group related functions, classes, and methods together to accomplish larger tasks.
- Build modules and packages that can be used by other applications.
- Make use of external packages in your own code.

Key Vocab

- **Module**: a file containing Python definitions and statements. A module's functions, classes, and global variables can be accessed by other modules.
- **Package**: a collection of modules that can be accessed as a group using the package name.
- **import** : the Python keyword used to access data from other packages and modules inside of the current module.
- **PyPI**: the **Python Package Index**. A repository of Python packages that can be downloaded and made available to your application.
- **pip** : the command line tool used to download packages from PyPI. **pip** is installed on your computer automatically when you download Python.
- **Virtual Environment**: a collection of modules, packages, and scripts that can be activated or deactivated at any time.
- **Pipenv**: a virtual environment tool that uses **pip** to manage the modules, packages, and scripts that you intend to use in your application.

Introduction

There are a number of outside resources we can leverage in working with our Python applications. For example, **pytest** is a Python-specific library, or **package**. We leverage this testing tool for behavior-driven testing. As we start to build out more complex applications, it's standard to

utilize packages and other conventions for setting up Python applications. This will not only save time, but will also offer a common template that is familiar to Python developers.

We'll cover topics including:

- What is a module?
- What is a package?
- How does `import` allow me to expand the functionality of my application?
- How do I find packages on PyPI?
- How is my virtual environment set up when I run `pipenv install` ?

In the next couple of lessons, we'll explain these tools and concepts that have been designed for efficiency.

Resources

- [Python 3 Documentation](https://docs.python.org/3/) ➞ [\(https://docs.python.org/3/\)](https://docs.python.org/3/)
- [Modules - Python](https://docs.python.org/3/tutorial/modules.html) ➞ [\(https://docs.python.org/3/tutorial/modules.html\)](https://docs.python.org/3/tutorial/modules.html)
- [Python Modules and Packages - An Introduction - Real Python](https://realpython.com/python-modules-packages/) ➞ [\(https://realpython.com/python-modules-packages/\)](https://realpython.com/python-modules-packages/)
- [Find, install, and publish Python packages with the Python Package Index - PyPI](https://pypi.org/) ➞ [\(https://pypi.org/\)](https://pypi.org/)
- [Pipenv: Python Dev Workflow for Humans - Pipenv](https://pipenv.pypa.io/en/latest/) ➞ [\(https://pipenv.pypa.io/en/latest/\)](https://pipenv.pypa.io/en/latest/)